

Sistem Terdistribusi dan Terdesentralisasi - #6

Bagas Triaji

Distributed File System

Distributed File System adalah suatu sistem penyimpanan berkas yang dirancang untuk memungkinkan data disimpan, dikelola, dan diakses secara terkoordinasi pada beberapa node komputer yang terhubung dalam suatu jaringan. Berbeda dengan sistem berkas tradisional yang terpusat pada satu server, DFS mendistribusikan data ke berbagai lokasi fisik sehingga pengguna dapat mengakses berkas seolah-olah semuanya berada dalam satu ruang penyimpanan terpadu.

Tujuan

Tujuan utama dari DFS adalah menyediakan layanan penyimpanan yang skalabel, tangguh, dan efisien terhadap pertumbuhan volume data maupun jumlah pengguna. Dengan memanfaatkan replikasi dan mekanisme redundansi, DFS juga memastikan ketersediaan data tetap terjaga meskipun salah satu node mengalami kegagalan.

Karakteristik

1. **Transparansi**

Pengguna tidak perlu mengetahui lokasi fisik data berada. Semua berkas terlihat seperti berada pada satu sistem yang sama.

2. **Reliabilitas dan Fault Tolerance**

Replikasi data dan teknik redundansi digunakan untuk mencegah kehilangan data akibat kegagalan sistem.

3. **Scalability**

Kapasitas penyimpanan dan kemampuan sistem dapat ditingkatkan dengan menambahkan node baru tanpa mengganggu layanan yang berjalan.

4. **Konsistensi Data**

Sistem mengatur agar setiap perubahan pada berkas direplikasi dan disinkronisasikan ke seluruh node yang menyimpan salinannya, sesuai kebijakan konsistensi yang diterapkan.

5. **Keamanan**

Mekanisme autentikasi, otorisasi, dan enkripsi diterapkan untuk melindungi data dari akses tidak sah.

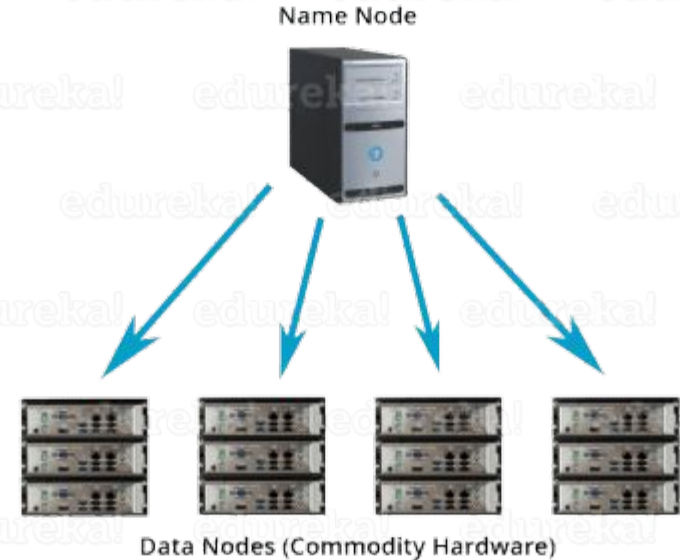
Contoh DFS

- HDFS
- Google FS
- Ceph
- GlusterFS

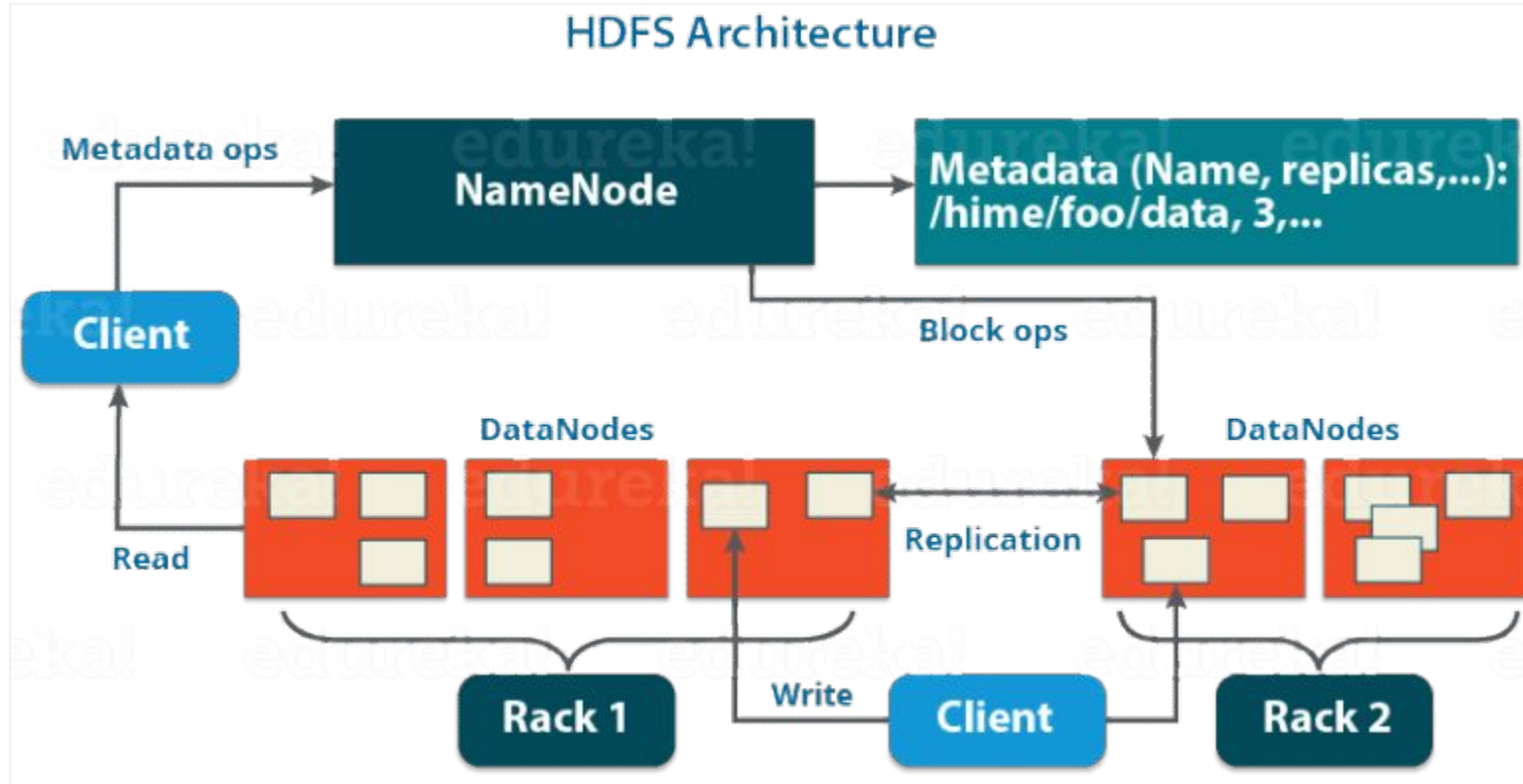
Hadoop Distributed File System (HDFS)

adalah sistem file berstruktur blok di mana setiap file dibagi menjadi beberapa blok dengan ukuran yang telah ditentukan sebelumnya.

Blok-blok ini disimpan di satu atau beberapa mesin cluster. HDFS menggunakan arsitektur Master/Slave, di mana sebuah cluster terdiri dari satu NameNode (Master node) dan node lainnya adalah DataNodes (Slave node).



Arsitektur HDFS



NameNode

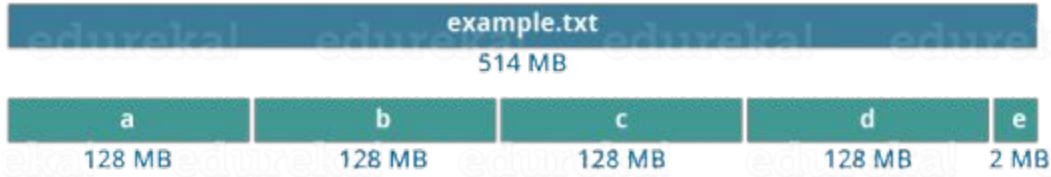
- Memelihara dan mengelola Data Node dan memberikan tugas kepada mereka.
- Namenode tidak berisi data file yang sebenarnya.
- Menyimpan metadata dari data aktual seperti Nama File, jalur, jumlah blok data, ID blok, lokasi blok, jumlah replika, dan informasi terkait *slave* lainnya.
- Mengelola semua permintaan (baca, tulis) klien untuk file data aktual.
- Mencatat setiap perubahan yang terjadi pada metadata sistem file. Misalnya, jika file dihapus dalam HDFS, NameNode akan segera merekamnya di EditLog.

DataNode

- Bertanggung jawab untuk menyimpan data aktual.
- Berdasar instruksi dari Namenode, ia dapat melakukan operasi seperti pembuatan/replikasi/penghapusan blok data.
- Ketika salah satu Datanode down maka tidak akan berpengaruh pada cluster Hadoop karena replikasi.
- Semua Datanodes disinkronkan di cluster Hadoop sedemikian rupa sehingga mereka dapat berkomunikasi satu sama lain untuk berbagai operasi.

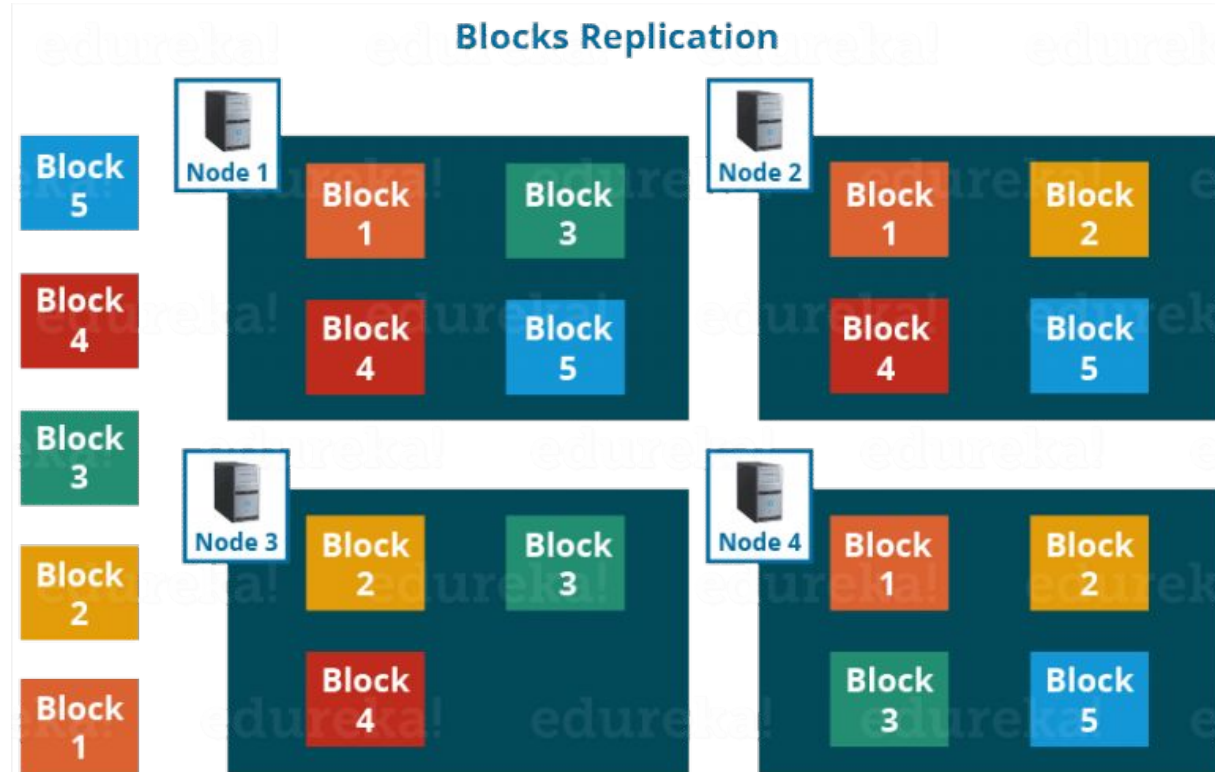
Blocks

HDFS menyimpan setiap file sebagai blok yang tersebar di seluruh cluster Apache Hadoop. Ukuran default setiap blok adalah 128 MB (dapat diubah/disesuaikan).



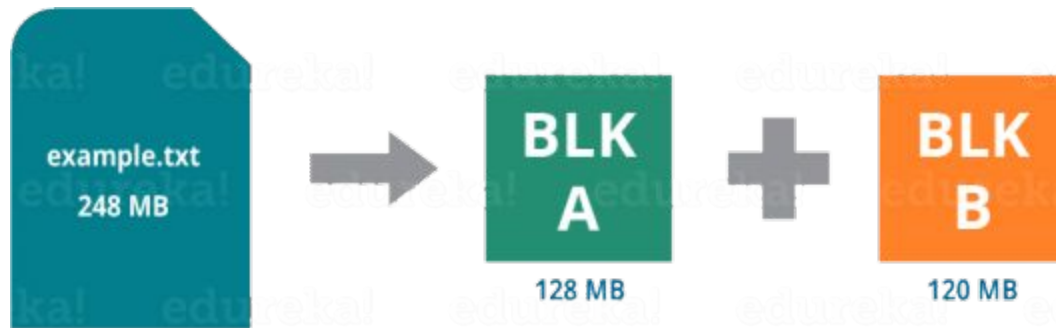
Block Replication

- Blok juga direplikasi untuk fault tolerance
- Nilai umum (default) replication factor adalah 3
- Apabila menyimpan file sebesar 100 MB, maka akan dikali 3 sehingga memakan tempat 300 MB



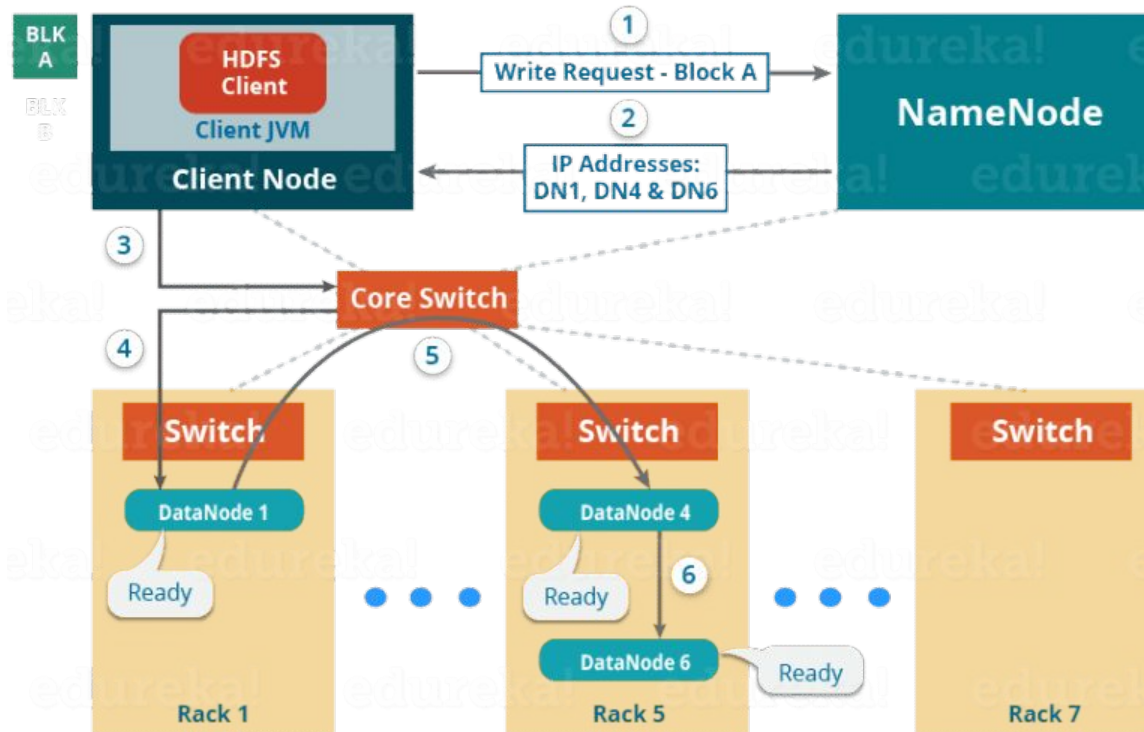
Operasi Write/Read

Contoh:

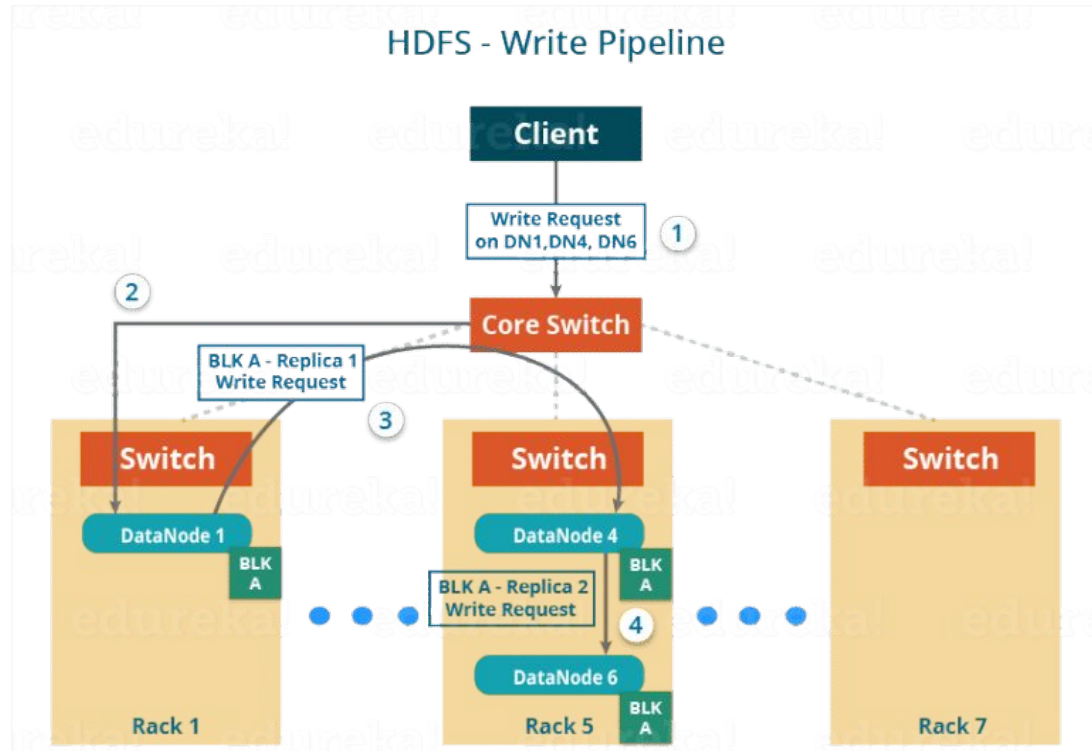


Write: Block A

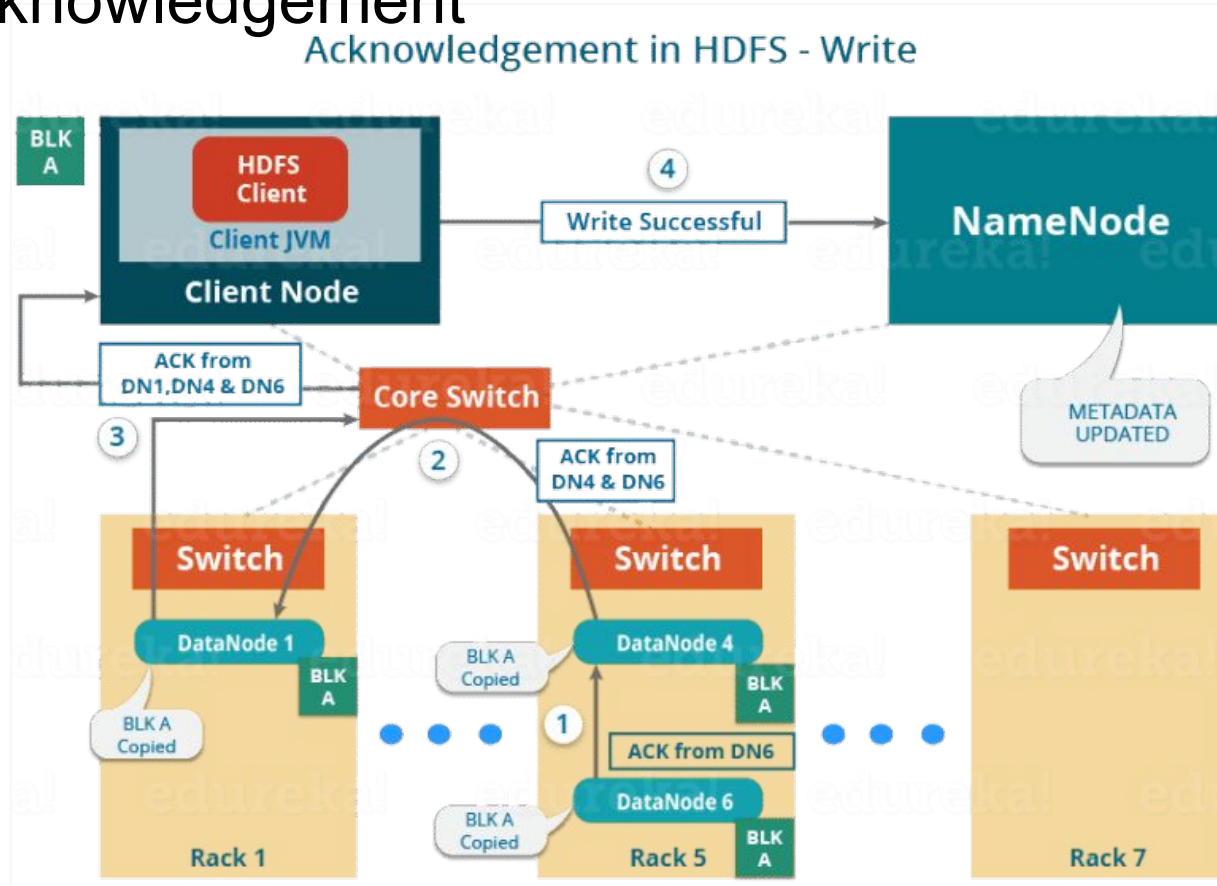
Setting up HDFS - Write Pipeline



Write: Block A



Write: Acknowledgement



Read

